Unidad 4

Objetos

Lo mínimo de objetos para el proyecto T4.1, objetos literales, propiedades, arrays de objetos, objetos como parámetro y copias que no mutan el original.

Un **objeto** agrupa en una sola variable varios datos que van juntos, cada uno con su nombre. En vez de tener `nombre`, `precio` y `unidades` sueltos, se tiene un producto con esas tres **propiedades**. Es la pieza básica de la **programación orientada a objetos**.

Esta página es sólo una introducción, con lo justo para el proyecto T4.1: crear objetos, leerlos, guardarlos en arrays y copiarlos sin cambiar el original.

Objetos literales

Un objeto se escribe entre llaves, con pares `clave: valor` separados por comas. La clave es el nombre de la propiedad; el valor, cualquier dato: número, texto, array u otro objeto. `{}` es un objeto vacío.

```
const producto = {
  nombre: 'Pan',
  precio: 1.2,
  unidades: 2,
};

console.log(producto);   // → { nombre: 'Pan', precio: 1.2, unidades: 2 }
```

Una función puede crear un objeto y devolverlo. Si una variable se llama igual que la propiedad, basta con su nombre:

```
function crearPunto(x, y) {
  return { x, y };       // igual que { x: x, y: y }
}
```

```
console.log(crearPunto(3, 4));    // → { x: 3, y: 4 }
```

Leer y cambiar propiedades

Se accede a una propiedad con un punto. Asignarle un valor la cambia, o la crea si no existía. Leer una que no existe da `undefined`, no un error:

```
console.log(producto.nombre);     // → Pan
producto.unidades = 3;            // cambia una propiedad
producto.oferta = true;           // añade una nueva
console.log(producto.peso);       // → undefined
```

`const` impide asignar otro objeto a la variable, pero no cambiar sus propiedades: pasa lo mismo que con los arrays.

Con corchetes, el nombre de la propiedad es un texto, que puede estar en una variable. Sirve cuando la propiedad se decide al ejecutar el programa:

```
const propiedad = 'precio';
console.log(producto[propiedad]);   // → 1.2
console.log(producto['nombre']);    // → Pan
```

Arrays de objetos

Una propiedad puede ser un array, y un array puede guardar objetos. Así se representa una lista de cosas con varios datos cada una:

```
const pedido = {
  cliente: 'Ana',
  productos: [
    { nombre: 'Pan', precio: 1.2, unidades: 2 },
    { nombre: 'Leche', precio: 0.9, unidades: 6 },
  ],
};
```

```
console.log(pedido.productos.length);      // → 2
console.log(pedido.productos[1].nombre);   // → Leche
```

Los métodos de la unidad anterior funcionan igual: la callback recibe cada objeto y lee sus propiedades.

```
const baratos = pedido.productos.filter((p) => p.precio < 1);
console.log(baratos);   // → [{ nombre: 'Leche', precio: 0.9, unidades: 6 }]
```

Objetos como parámetro

Un objeto se pasa a una función igual que un array: se copia la **referencia**, así que la función trabaja sobre el mismo objeto y los cambios se ven fuera (ver «Arrays, conjuntos y mapas como parámetro» en Funciones):

```
function duplicarUnidades(producto) {
  producto.unidades = producto.unidades * 2;
}

const pan = { nombre: 'Pan', precio: 1.2, unidades: 2 };
duplicarUnidades(pan);
console.log(pan.unidades);   // → 4
```

Copiar sin mutar

Una función **no muta** lo que recibe cuando devuelve un objeto nuevo con el cambio y deja el original como estaba. El cambio se guarda fuera, sustituyendo el objeto antiguo por el nuevo.

Para copiar un objeto se usa `...`, el mismo operador que copia arrays en la unidad 3. Las propiedades que van detrás sustituyen a las copiadas:

```
const pan = { nombre: 'Pan', precio: 1.2, unidades: 2 };
const masPan = { ...pan, unidades: 4 };   // copia y cambia unidades

console.log(pan);      // → { nombre: 'Pan', precio: 1.2, unidades: 2 }
console.log(masPan);   // → { nombre: 'Pan', precio: 1.2, unidades: 4 }
```

La copia es **superficial**: si una propiedad es un array u otro objeto, se copia su referencia, no su contenido. La copia y el original comparten el mismo array:

```
const pedido = { cliente: 'Ana', productos: [] };
const copia = { ...pedido };

copia.productos.push({ nombre: 'Pan', precio: 1.2, unidades: 2 });
console.log(pedido.productos.length);   // → 1: el original también cambia
```

Para no tocar el original, el array también se crea nuevo:

```
const pedido = { cliente: 'Ana', productos: [] };
const conPan = {
  ...pedido,
  productos: [...pedido.productos, { nombre: 'Pan', precio: 1.2 }],
};

console.log(pedido.productos.length);   // → 0
console.log(conPan.productos.length);   // → 1
```

`map`, `filter` y `slice` devuelven arrays nuevos; `push`, `splice` y `sort` cambian el array sobre el que se llaman. Pero un array nuevo puede guardar los mismos objetos que el original: para cambiar uno, se pone en su lugar una copia con el cambio.

```
const pedido = {
  cliente: 'Ana',
  productos: [
    { nombre: 'Pan', precio: 1.2, unidades: 2 },
    { nombre: 'Leche', precio: 0.9, unidades: 6 },
  ],
};

const panRebajado = {
  ...pedido,
  productos: pedido.productos.map((p) =>
    p.nombre === 'Pan' ? { ...p, precio: 1 } : p),
};
```

```
console.log(pedido.productos[0].precio);        // → 1.2
console.log(panRebajado.productos[0].precio);   // → 1
```

Comparar objetos

`===` con dos objetos no compara su contenido: dice si son **el mismo** objeto. Dos objetos con las mismas propiedades son distintos, y una copia es siempre otro objeto. Así se comprueba que una función ha devuelto un objeto nuevo:

```
const a = { precio: 1 };
const b = { precio: 1 };

console.log(a === b);          // → false: son dos objetos
console.log(a === a);          // → true
console.log({ ...a } === a);   // → false: la copia es otro objeto
```